

Dossier d'Architecture de Base de Données - Spécification Finale

Projet : Système Multi-Tenant de Digitalisation Intégré pour Centres de Formation

Client : VITALIS CENTER EUP

Date : 06 Juillet 2026

Version : 3.0 (Livrable Définitif Révisé)

Statut : Prêt pour Téléchargement et Déploiement Production

Ce document consolide l'ensemble des structures de données relationnelles nécessaires au fonctionnement global de la plateforme, incluant la gestion complète des 12 tables, la planification temporelle des séances, le suivi d'émargement et les politiques de sécurité *Row Level Security (RLS)* sur Supabase.

1. Cartographie d'Architecture (Les 12 Tables Système)

Le système repose sur un modèle à 12 tables interdépendantes, assurant le cloisonnement multi-tenant, le suivi pédagogique complet, la planification temporelle et un journal d'audit immuable.

N°	Nom de la Table	Rôle Métier Principal	Type de Relation Majeure
1	etablisements	Cloisonnement étanche des structures (Multi-Tenant).	Pivot racine (1:N avec utilisateurs et formations)
2	utilisateurs	Registre centralisé des comptes (RBAC multi-rôles).	N:1 vers etablisements
3	formations	Catalogue des filières et parcours certifiants.	N:1 vers etablisements
4	modules	Unités d'enseignement structurées à coefficients.	N:1 vers formations (Cascade)
5	cours	Supports et chapitres de cours textuels ou documentaires.	N:1 vers modules (Cascade)
6	seances_formation	Planification temporelle des cours physiques ou virtuels.	N:1 vers cours, modules et formateurs
7	presences_seances	Suivi d'émargement électronique et de ponctualité.	Pivot N:N (utilisateurs <-> seances_formation)
8	progression_cours	Suivi de l'état de complétion individuel des chapitres.	Pivot N:N (utilisateurs <-> cours)

N°	Nom de la Table	Rôle Métier Principal	Type de Relation Majeure
9	evaluations	Définition des examens, épreuves et contrôles continus.	N:1 vers modules (Cascade)
10	notes	Registre des notes (Strictement bornées de 0.00 à 20.00).	N:1 unique (utilisateurs, evaluations, formateurs)
11	certificats	Titres de compétences sécurisés par hachage unique.	N:1 vers utilisateurs et formations
12	table_audit	Registre immuable de sécurité et de traçabilité.	Historique autonome (Relations faibles vers utilisateurs)

2. Script DDL PostgreSQL d'Initialisation Production

Ce script SQL contient l'ensemble des types énumérés, des tables avec clés étrangères et des contraintes d'intégrité métier nécessaires pour configurer directement l'instance Supabase.

```
-- Extension UUID
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

-- Énumérations RBAC, Séances et Émargement
CREATE TYPE utilisateur_role AS ENUM ('ADMIN_CENTRE', 'ADMIN_ETABLISSEMENT',
'FORMATEUR', 'PERSONNEL_ADMINISTRATIF', 'APPRENANT');
CREATE TYPE type_seance AS ENUM ('THEORIQUE', 'PRATIQUE', 'ATELIER', 'EVALUATION');
CREATE TYPE statut_presence AS ENUM ('PRESENT', 'ABSENT', 'RETARD', 'JUSTIFIE');

-- 1. Table des établissements
CREATE TABLE etablisements (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  nom VARCHAR(255) NOT NULL,
  code_antenne VARCHAR(50) UNIQUE NOT NULL,
  adresse TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Table des utilisateurs
CREATE TABLE utilisateurs (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  etablisement_id UUID NOT NULL REFERENCES etablisements(id) ON DELETE RESTRICT,
  nom VARCHAR(100) NOT NULL,
  prenom VARCHAR(100) NOT NULL,
  email VARCHAR(255) UNIQUE NOT NULL,
  mot_de_passe_hash VARCHAR(255) NOT NULL,
  role utilisateur_role NOT NULL,
  est_actif BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

```

);

-- 3. Table des formations
CREATE TABLE formations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    etablissement_id UUID NOT NULL REFERENCES etablissements(id) ON DELETE RESTRICT,
    titre VARCHAR(255) NOT NULL,
    description TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Table des modules
CREATE TABLE modules (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    formation_id UUID NOT NULL REFERENCES formations(id) ON DELETE CASCADE,
    titre VARCHAR(255) NOT NULL,
    ordre INT NOT NULL,
    coefficient NUMERIC(4,2) DEFAULT 1.00,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 5. Table des cours
CREATE TABLE cours (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    module_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,
    titre VARCHAR(255) NOT NULL,
    contenu_text TEXT,
    url_document_s3 VARCHAR(512),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 6. Table des séances de formation
CREATE TABLE seances_formation (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    module_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,
    cours_id UUID REFERENCES cours(id) ON DELETE SET NULL,
    formateur_id UUID NOT NULL REFERENCES utilisateurs(id) ON DELETE RESTRICT,
    titre_activite VARCHAR(255) NOT NULL,
    type_session type_seance NOT NULL DEFAULT 'THEORIQUE',
    date_heure_debut TIMESTAMP WITH TIME ZONE NOT NULL,
    date_heure_fin TIMESTAMP WITH TIME ZONE NOT NULL,
    salle_ou_lien VARCHAR(255),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_dates_coherentes CHECK (date_heure_fin > date_heure_debut)
);

-- 7. Table des présences aux séances (Émargement)
CREATE TABLE presences_seances (
    seance_id UUID NOT NULL REFERENCES seances_formation(id) ON DELETE CASCADE,
    apprenant_id UUID NOT NULL REFERENCES utilisateurs(id) ON DELETE CASCADE,
    statut_statut_presence NOT NULL DEFAULT 'ABSENT',
    remarque_justification TEXT,
    mis_a_jour_a TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (seance_id, apprenant_id)
);

```

```

);

-- 8. Table de progression des cours
CREATE TABLE progression_cours (
    apprenant_id UUID NOT NULL REFERENCES utilisateurs(id) ON DELETE CASCADE,
    cours_id UUID NOT NULL REFERENCES cours(id) ON DELETE CASCADE,
    termine BOOLEAN DEFAULT FALSE,
    date_termination TIMESTAMP WITH TIME ZONE,
    PRIMARY KEY (apprenant_id, cours_id)
);

-- 9. Table des évaluations
CREATE TABLE evaluations (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    module_id UUID NOT NULL REFERENCES modules(id) ON DELETE CASCADE,
    titre VARCHAR(255) NOT NULL,
    note_maximale NUMERIC(4,2) DEFAULT 20.00,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 10. Table des notes
CREATE TABLE notes (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    evaluation_id UUID NOT NULL REFERENCES evaluations(id) ON DELETE RESTRICT,
    apprenant_id UUID NOT NULL REFERENCES utilisateurs(id) ON DELETE CASCADE,
    formateur_id UUID NOT NULL REFERENCES utilisateurs(id) ON DELETE RESTRICT,
    note_obtenue NUMERIC(4,2) NOT NULL CHECK (note_obtenue >= 0 AND note_obtenue <=
20),
    date_notation TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT unique_note_apprenant_evaluation UNIQUE (evaluation_id, apprenant_id)
);

-- 11. Table des certificats
CREATE TABLE certificats (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    apprenant_id UUID NOT NULL REFERENCES utilisateurs(id) ON DELETE RESTRICT,
    formation_id UUID NOT NULL REFERENCES formations(id) ON DELETE RESTRICT,
    numero_serie VARCHAR(50) UNIQUE NOT NULL,
    hash_verification VARCHAR(64) UNIQUE NOT NULL,
    url_pdf_s3 VARCHAR(512) NOT NULL,
    moyenne_generale NUMERIC(4,2) NOT NULL,
    date_emission TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 12. Table d'audit immuable
CREATE TABLE table_audit (
    id BIGSERIAL PRIMARY KEY,
    timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    adresse_ip VARCHAR(45) NOT NULL,
    utilisateur_id UUID REFERENCES utilisateurs(id) ON DELETE SET NULL,
    action_effectuee VARCHAR(100) NOT NULL,
    table_cible VARCHAR(100) NOT NULL,
    etat_avant JSONB,
    etat_apres JSONB
);

```

```
);

-- Index d'optimisation de performance pour requêtes < 300ms
CREATE INDEX idx_utilisateurs_etablissement ON utilisateurs(etablissement_id);
CREATE INDEX idx_seances_debut_fin ON seances_formation(date_heure_debut,
date_heure_fin);
CREATE INDEX idx_notes_apprenant_evaluation ON notes(apprenant_id, evaluation_id);
CREATE INDEX idx_certificats_hash ON certificats(hash_verification);
```

3. Schéma de Déclaration pour Prisma ORM (schema.prisma)

Ce schéma sert à configurer le client Prisma dans l'écosystème NestJS, en associant les types natifs et en assurant la liaison complète des objets relationnels.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum UsuarioRole {
  ADMIN_CENTRE
  ADMIN_ETABLISSEMENT
  FORMATEUR
  PERSONNEL_ADMINISTRATIF
  APPRENANT
}

enum TypeSeance {
  THEORIQUE
  PRATIQUE
  ATELIER
  EVALUATION
}

enum StatutPresence {
  PRESENT
  ABSENT
  RETARD
  JUSTIFIE
}

model Etablissement {
  id          String      @id @default(uuid()) @db.Uuid
  nom         String
  codeAntenne String      @unique @map("code_antenne")
  adresse     String?
  utilisateurs Utilisateur[]
  formations  Formation[]
}
```

```

    createdAt      DateTime      @default(now()) @map("created_at")
    updatedAt      DateTime      @updatedAt @map("updated_at")

    @@map("etablissements")
}

model Utilisateur {
    id              String          @id @default(uuid()) @db.Uuid
    etablissementId String          @db.Uuid @map("etablissement_id")
    etablissement   Etablissement   @relation(fields: [etablissementId],
references: [id], onUpdate: Restrict)
    nom             String
    prenom          String
    email           String          @unique
    motDePasseHash String          @map("mot_de_passe_hash")
    role            UsuarioRole
    estActif        Boolean          @default(true) @map("est_actif")
    notesEvaluees   Note[]          @relation("FormateurNotes")
    notesRecues     Note[]          @relation("ApprenantNotes")
    certificats     Certificat[]
    progressions    ProgressionCours[]
    seancesDispensees SeanceFormation[] @relation("FormateurSeances")
    presencesSeances PresencesSeance[]
    createdAt      DateTime          @default(now()) @map("created_at")
    updatedAt      DateTime          @updatedAt @map("updated_at")

    @@map("utilisateurs")
}

model Formation {
    id              String          @id @default(uuid()) @db.Uuid
    etablissementId String          @db.Uuid @map("etablissement_id")
    etablissement   Etablissement @relation(fields: [etablissementId], references:
[id], onUpdate: Restrict)
    titre           String
    description      String?
    modules          Module[]
    certificats     Certificat[]
    createdAt      DateTime          @default(now()) @map("created_at")

    @@map("formations")
}

model Module {
    id              String          @id @default(uuid()) @db.Uuid
    formationId     String          @db.Uuid @map("formation_id")
    formation       Formation       @relation(fields: [formationId], references:
[id], onDelete: Cascade)
    titre           String
    ordre           Int
    coefficient      Decimal          @default(1.00) @db.Decimal(4, 2)
    cours           Cours[]
    evaluations      Evaluation[]
    seancesFormation SeanceFormation[]

```

```

    createdAt      DateTime      @default(now()) @map("created_at")

    @@map("modules")
}

model Cours {
    id              String          @id @default(uuid()) @db.Uuid
    moduleId        String          @db.Uuid @map("module_id")
    module          Module          @relation(fields: [moduleId], references: [id],
onDelete: Cascade)
    titre          String
    contenuText     String?         @map("contenu_text")
    urlDocumentS3   String?         @map("url_document_s3")
    progressions    ProgressionCours[]
    seancesAssociees SeanceFormation[]
    createdAt      DateTime      @default(now()) @map("created_at")

    @@map("cours")
}

model SeanceFormation {
    id              String          @id @default(uuid()) @db.Uuid
    moduleId        String          @db.Uuid @map("module_id")
    module          Module          @relation(fields: [moduleId], references: [id],
onDelete: Cascade)
    coursId         String?         @db.Uuid @map("cours_id")
    cours           Cours?         @relation(fields: [coursId], references: [id],
onDelete: SetNull)
    formateurId     String          @db.Uuid @map("formateur_id")
    formateur       Utilisateur     @relation("FormateurSeances", fields:
[formateurId], references: [id], onUpdate: Restrict)
    titreActivite   String          @map("titre_activite")
    typeSession     TypeSeance      @default(THEORIQUE) @map("type_session")
    dateHeureDebut  DateTime        @map("date_heure_debut")
    dateHeureFin    DateTime        @map("date_heure_fin")
    salleOuLien     String?         @map("salle_ou_lien")
    presences       PresencesSeance[]
    createdAt      DateTime      @default(now()) @map("created_at")

    @@map("seances_formation")
}

model PresencesSeance {
    seanceId        String          @db.Uuid @map("seance_id")
    seance          SeanceFormation @relation(fields: [seanceId], references:
[id], onDelete: Cascade)
    apprenantId     String          @db.Uuid @map("apprenant_id")
    apprenant       Utilisateur     @relation(fields: [apprenantId], references:
[id], onDelete: Cascade)
    statut          StatutPresence @default(ABSENT)
    remarqueJustification String?   @map("remarque_justification")
    misAJourA       DateTime        @default(now()) @map("mis_a_jour_a")

    @@id([seanceId, apprenantId])
}

```

```

    @@map("presences_seances")
}

model ProgressionCours {
    apprenantId      String      @db.Uuid @map("apprenant_id")
    apprenant        Utilisateur @relation(fields: [apprenantId], references: [id],
onDelete: Cascade)
    coursId          String      @db.Uuid @map("cours_id")
    cours            Cours       @relation(fields: [coursId], references: [id],
onDelete: Cascade)
    termine          Boolean      @default(false)
    dateTerminaison  DateTime?    @map("date_terminaison")

    @@id([apprenantId, coursId])
    @@map("progression_cours")
}

model Evaluation {
    id                String      @id @default(uuid()) @db.Uuid
    moduleId          String      @db.Uuid @map("module_id")
    module            Module      @relation(fields: [moduleId], references: [id], onDelete:
Cascade)
    titre             String
    noteMaximale      Decimal      @default(20.00) @db.Decimal(4, 2) @map("note_maximale")
    notes             Note[]
    createdAt         DateTime @default(now()) @map("created_at")

    @@map("evaluations")
}

model Note {
    id                String      @id @default(uuid()) @db.Uuid
    evaluationId      String      @db.Uuid @map("evaluation_id")
    evaluation         Evaluation @relation(fields: [evaluationId], references: [id],
onUpdate: Restrict)
    apprenantId       String      @db.Uuid @map("apprenant_id")
    apprenant         Utilisateur @relation("ApprenantNotes", fields: [apprenantId],
references: [id], onDelete: Cascade)
    formateurId       String      @db.Uuid @map("formateur_id")
    formateur         Utilisateur @relation("FormateurNotes", fields: [formateurId],
references: [id], onUpdate: Restrict)
    noteObtenue       Decimal      @db.Decimal(4, 2) @map("note_obtenue")
    dateNotation       DateTime    @default(now()) @map("date_notation")

    @@unique([evaluationId, apprenantId])
    @@map("notes")
}

model Certificat {
    id                String      @id @default(uuid()) @db.Uuid
    apprenantId       String      @db.Uuid @map("apprenant_id")
    apprenant         Utilisateur @relation(fields: [apprenantId], references: [id],
onUpdate: Restrict)
    formationId       String      @db.Uuid @map("formation_id")

```



```

    formation      Formation    @relation(fields: [formationId], references: [id],
onUpdate: Restrict)
    numeroSerie    String       @unique @map("numero_serie")
    hashVerification String     @unique @map("hash_verification")
    urlPdfS3       String       @map("url_pdf_s3")
    moyenneGenerale Decimal     @db.Decimal(4, 2) @map("moyenne_generale")
    dateEmission   DateTime     @default(now()) @map("date_emission")

    @@map("certificats")
}

model TableAudit {
  id          BigInt    @id @default(autoincrement())
  timestamp   DateTime  @default(now())
  adresseIp   String    @map("adresse_ip")
  utilisateurId String?  @db.Uuid @map("utilisateur_id")
  actionEffectuee String  @map("action_effectuee")
  tableCible  String    @map("table_cible")
  etatAvant   Json?     @map("etat_avant")
  etatApres   Json?     @map("etat_apres")

  @@map("table_audit")
}

```

4. Politiques Row Level Security (RLS) Supabase

Ces directives assurent le cloisonnement strict Multi-Tenant au niveau de la base de données. L'accès aux séances et l'émargement électronique obéissent à des contraintes basées sur l'utilisateur connecté (***auth.uid()***).

```

-- Activation de la RLS sur le domaine temporel
ALTER TABLE seances_formation ENABLE ROW LEVEL SECURITY;
ALTER TABLE presences_seances ENABLE ROW LEVEL SECURITY;

-- RLS: Visibilité des séances limitée aux membres de l'établissement concerné
CREATE POLICY politique_lecture_seances ON seances_formation
  FOR SELECT
  USING (
    EXISTS (
      SELECT 1 FROM utilisateurs
      WHERE utilisateurs.id = auth.uid()
      AND utilisateurs.etablissement_id = (
        SELECT etablissement_id FROM modules
        JOIN formations ON formations.id = modules.formation_id
        WHERE modules.id = seances_formation.module_id
      )
    )
  );

-- RLS: Droits d'émargement réservés au formateur assigné ou à l'administration
CREATE POLICY politique_gestion_presences ON presences_seances
  FOR ALL

```

```
    USING (
        EXISTS (
            SELECT 1 FROM seances_formation sf
            WHERE sf.id = presences_seances.seance_id
            AND (sf.formateur_id = auth.uid() OR EXISTS (
                SELECT 1 FROM utilisateurs u
                WHERE u.id = auth.uid() AND u.role IN ('ADMIN_ETABLISSEMENT',
'ADMIN_CENTRE')
            ))
        )
    );
```